

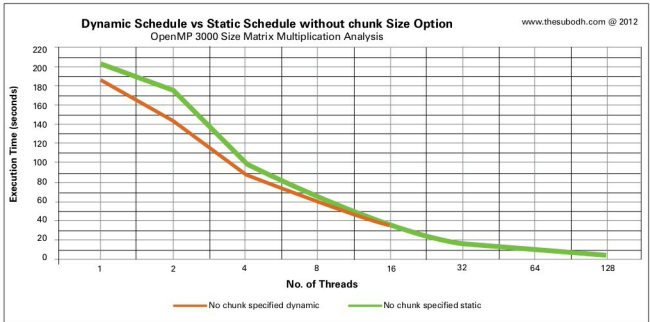


Figure 4: Evaluating the performance of dynamic schedule vs static schedules

of worker threads to be invoked is provided through the `omp_set_num_threads()` function. If not specified, OpenMP will invoke one thread for each CPU core in the system. A 2D array is initialised, followed by memory allocation using `malloc()`. Now we need to analyse the result of matrix multiplication because it could contain erroneous results, due to race conditions, which are common in parallel programming. So I have chosen static data for matrix elements, to verify the results easily using serial code. Here, I have assigned $m1$ and $m2$ matrices with '1' as the element value, so that each element in mul matrix would end up as the same value given by the user as per the matrix size. For example, if the matrix size is 3000, then $mul[0][0]$ would be 3000 (if the operation is error-free). Now, assigning values to elements of $m1$ and $m2$ will take long, especially if the matrix is large. To circumvent this, I have used a conditional *pragma* directive here. If the matrix size is over 2000, the *for* loop will become parallelised. Parallel assignment of values to $m1$ and $m2$ improves performance, because the assignment operators are independent of each other. Never use function calls in

parallel unless they are parallel too.

The operation time calculated is only for the multiplication section of the code. OpenMP provides the function `omp_get_wtime()` to report the current time. The difference between the start time and the end time is the time required to perform the parallel multiplication operation.

The parallel region with global shared variables for matrix data and the global private variable for the thread number is followed by a *parallel for* region. The *Pragma for* statement applies only to one loop; a nested *for* loop parallelism is achieved using multiple *pragma omp* directives. By default, in the *pragma for* directive, the variable 'i' will become private because I explicitly declared them as private to make the code more understandable. The default type in *pragma omp* parallel constructs is shared unless specified; you can change it by using *default(private)* in the *pragma omp* parallel statement.

The *nowait* clause in the *pragma for* statement will remove the implied barrier at the end of the parallel region, and continue to execute the next statement in the *pragma*

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. info@technoinfotech.com